System Design Document — TaskAnalyzer

Bootcamp III — Fase 3 (Entrega Final)
Stella Cardoso de Campos Mascaro — Ciência da Computação — Polo Asa Norte

1. Visão geral

O TaskAnalyzer é um módulo Python que analisa uma lista de tarefas e retorna métricas de produtividade: totais, tempo médio de conclusão (em horas) e taxa de atraso (%), de forma global e por prioridade (baixa, media, alta). O projeto segue Spec-Driven Development (SDD): a especificação e as regras de contexto (CONTEXT_RULES.md) guiam o código gerado por IA, que é validado por testes automatizados (pytest) e homologado por revisão humana.

2. Arquitetura do repositório

```
sdd-taskanalyzer/
|- .github/workflows/ci.yml     # Pipeline CI (GitHub Actions)
|- Dockerfile                   # Empacotamento (python:3.11-slim)
|- docker-compose.yml           # Orquestração da suíte de testes
|- CONTEXT_RULES.md             # Governança de IA (Fase 1)
|- specs/
|   |- task_analyzer_spec.md    # Contrato SDD
|   |- requirements.yaml        # Requisitos estruturados
|   `- system_design_document.pdf
|- tests/test_harness.py        # Test Harness (pytest)
|- src/task_analyzer.py         # Código gerado via IA e homologado
|- requirements.txt
`- README.md
```

3. Componentes

Componente	Responsabilidade
analyze_tasks	Ponto de entrada público: valida as tarefas, agrega métricas e retorna o dicionário de resultados.
TaskValidationError	Exceção específica (subclasse de ValueError) para dados inconsistentes.
_validar_tarefa / _validar_identificacao / _validar_dominio / _calcular_conclusao	Validação por responsabilidade única (SRP): campos, tipos, domínio e datas.
_media / _taxa_atraso / _resumir	Cálculos puros, com prevenção de divisão por zero (retornam 0.0).

4. Contrato de dados

Entrada (lista de dicionários): id (int ou str), titulo (str), prioridade (baixa | media | alta), data_criacao, prazo, status (pendente | concluida) e data_conclusao (obrigatória se concluida).

Saída: total_tarefas, total_concluidas, total_pendentes, tempo_medio_conclusao_horas (alias tempo_medio_conclusao_global), taxa_atraso_percentual (alias taxa_atraso_global) e indicadores_por_prioridade (as mesmas métricas para cada prioridade).

Regras: pendentes são ignoradas nas médias; atraso quando data_conclusao > prazo; data_conclusao < data_criacao dispara TaskValidationError; arredondamento em 2 casas; datas sem fuso tratadas como UTC.

5. Fluxo SDD com IA (Critical-in-the-Loop)

Etapa	Descrição
1. Geração	O assistente de IA recebe CONTEXT_RULES.md e a especificação e gera src/task_analyzer.py.
2. Barreira SDD	O código é confrontado com o contrato executável e com as 8 diretrizes e 8 proibições.
3. Validação humana	Execução do pytest, revisão de conformidade, qualidade, casos de borda e tratamento de erros.
4. Integração segura	Somente após a aprovação o código é commitado via Pull Request.

6. Pipeline CI/CD (GitHub Actions)

Arquivo .github/workflows/ci.yml. **Gatilhos**: push e pull request nas branches main e develop. **Ambiente**: ubuntu-latest com Python 3.11. **Passos**: checkout, configuração do Python (com cache de pip), instalação via requirements.txt e execução de `pytest tests/test_harness.py -v`. Um pipeline verde (green build) é pré-requisito para aprovar o merge.

7. Containerização (Docker)

O **Dockerfile** usa a imagem leve python:3.11-slim, instala as dependências, copia src/ e tests/ e define o pytest como comando padrão (CMD). O **docker-compose.yml** define o serviço *tests*, permitindo executar toda a suíte isolada com um único comando:

```
docker compose build
docker compose run --rm tests
```

O ambiente é imutável e reprodutível, eliminando o problema do "funciona na minha máquina".

8. Qualidade, segurança e conformidade

Aspecto	Decisão
Dependências	Somente biblioteca padrão; pytest apenas para testes (CONTEXT_RULES, proibição 1).
Dados	Processamento exclusivamente em memória, sem persistência (proibição 4); nenhum dado pessoal é armazenado.
Erros	Exceção específica com mensagem clara; sem except genérico.
Observabilidade	Logs com o módulo logging.
Rastreabilidade	Prompts registrados, commits atômicos, Pull Requests revisados e pipeline auditável.

9. Estratégia de testes

O Test Harness (tests/test_harness.py) traduz os cenários BDD: Cenário 1 (métricas corretas, global e por prioridade), Cenário 2 (TaskValidationError para datas inconsistentes e entradas inválidas) e casos de borda (lista vazia, somente pendentes, conclusão exatamente no prazo). Os testes rodam localmente, no GitHub Actions e no Docker, com o mesmo resultado esperado: 100% aprovados.